

The routine `newt` assumes that the typical values of all components of $\mathbf{x}$ and of $\mathbf{F}$ are of order unity, and it can fail if this assumption is badly violated. You should rescale the variables by their typical values before invoking `newt` if this problem occurs.

9.7.3 Multidimensional Secant Methods: Broyden's Method

Newton's method as implemented above is quite powerful, but it still has several disadvantages. One drawback is that the Jacobian matrix is needed. In many problems analytic derivatives are unavailable. If function evaluation is expensive, then the cost of finite difference determination of the Jacobian can be prohibitive.

Just as the quasi-Newton methods to be discussed in §10.9 provide cheap approximations for the Hessian matrix in minimization algorithms, there are quasi-Newton methods that provide cheap approximations to the Jacobian for zero finding. These methods are often called *secant methods*, since they reduce to the secant method (§9.2) in one dimension (see, e.g., [2]). The best of these methods still seems to be the first one introduced, *Broyden's method* [3].

Let us denote the approximate Jacobian by $\mathbf{B}$. Then the i th quasi-Newton step $\delta\mathbf{x}_i$ is the solution of

$$\mathbf{B}_i \cdot \delta\mathbf{x}_i = -\mathbf{F}_i \quad (9.7.15)$$

where $\delta\mathbf{x}_i = \mathbf{x}_{i+1} - \mathbf{x}_i$ (cf. equation 9.7.3). The quasi-Newton or secant condition is that $\mathbf{B}_{i+1}$ satisfy

$$\mathbf{B}_{i+1} \cdot \delta\mathbf{x}_i = \delta\mathbf{F}_i \quad (9.7.16)$$

where $\delta\mathbf{F}_i = \mathbf{F}_{i+1} - \mathbf{F}_i$. This is the generalization of the one-dimensional secant approximation to the derivative, $\delta F / \delta x$. However, equation (9.7.16) does not determine $\mathbf{B}_{i+1}$ uniquely in more than one dimension.

Many different auxiliary conditions to pin down $\mathbf{B}_{i+1}$ have been explored, but the best-performing algorithm in practice results from Broyden's formula. This formula is based on the idea of getting $\mathbf{B}_{i+1}$ by making the least change to $\mathbf{B}_i$ consistent with the secant equation (9.7.16). Broyden showed that the resulting formula is

$$\mathbf{B}_{i+1} = \mathbf{B}_i + \frac{(\delta\mathbf{F}_i - \mathbf{B}_i \cdot \delta\mathbf{x}_i) \otimes \delta\mathbf{x}_i}{\delta\mathbf{x}_i \cdot \delta\mathbf{x}_i} \quad (9.7.17)$$

You can easily check that $\mathbf{B}_{i+1}$ satisfies (9.7.16).

Early implementations of Broyden's method used the Sherman-Morrison formula, equation (2.7.2), to invert equation (9.7.17) analytically,

$$\mathbf{B}_{i+1}^{-1} = \mathbf{B}_i^{-1} + \frac{(\delta\mathbf{x}_i - \mathbf{B}_i^{-1} \cdot \delta\mathbf{F}_i) \otimes \delta\mathbf{x}_i \cdot \mathbf{B}_i^{-1}}{\delta\mathbf{x}_i \cdot \mathbf{B}_i^{-1} \cdot \delta\mathbf{F}_i} \quad (9.7.18)$$

Then, instead of solving equation (9.7.3) by, e.g., *LU* decomposition, one determined

$$\delta\mathbf{x}_i = -\mathbf{B}_i^{-1} \cdot \mathbf{F}_i \quad (9.7.19)$$

by matrix multiplication in $O(N^2)$ operations. The disadvantage of this method is that it cannot easily be embedded in a globally convergent strategy, for which the gradient of equation (9.7.4) requires $\mathbf{B}$, not $\mathbf{B}^{-1}$,

$$\nabla(\frac{1}{2}\mathbf{F} \cdot \mathbf{F}) \simeq \mathbf{B}^T \cdot \mathbf{F} \quad (9.7.20)$$

Accordingly, we implement the update formula in the form (9.7.17).

However, we can still preserve the $O(N^2)$ solution of (9.7.3) by using *QR* decomposition (§2.10) instead of *LU* decomposition. The reason is that because of the special form of equation (9.7.17), the *QR* decomposition of $\mathbf{B}_i$ can be updated into the *QR* decomposition of $\mathbf{B}_{i+1}$ in $O(N^2)$ operations (§2.10). All we need is an initial approximation $\mathbf{B}_0$ to start the ball rolling. It is often acceptable to start simply with the identity matrix, and then allow $O(N)$

updates to produce a reasonable approximation to the Jacobian. We prefer to spend the first N function evaluations on a finite difference approximation to initialize $\mathbf{B}$ via a call to `NRfdjac`.

Since $\mathbf{B}$ is not the exact Jacobian, we are not guaranteed that $\delta\mathbf{x}$ is a descent direction for $f = \frac{1}{2}\mathbf{F} \cdot \mathbf{F}$ (cf. equation 9.7.5). Thus the line search algorithm can fail to return a suitable step if $\mathbf{B}$ wanders far from the true Jacobian. In this case, we reinitialize $\mathbf{B}$ by another call to `NRfdjac`.

Like the secant method in one dimension, Broyden's method converges superlinearly once you get close enough to the root. Embedded in a global strategy, it is almost as robust as Newton's method, and often needs far fewer function evaluations to determine a zero. Note that the final value of $\mathbf{B}$ is *not* always close to the true Jacobian at the root, even when the method converges.

The routine `broydn`, given below, is very similar to `newt` in organization. The principal differences are the use of *QR* decomposition instead of *LU*, and the updating formula instead of directly determining the Jacobian. The remarks at the end of `newt` about scaling the variables apply equally to `broydn`.

roots_multidim.h

```
template <class T>
void broydn(VecDoub_IO &x, Bool &check, T &vecfunc) {
    Given an initial guess x[0..n-1] for a root in n dimensions, find the root by Broyden's
    method embedded in a globally convergent strategy. The vector of functions to be zeroed,
    called fvec[0..n-1] in the routine below, is returned by the user-supplied function or functor
    vecfunc. The routines NRfdjac and NRfmin from newt are used. The output quantity check
    is false on a normal return and true if the routine has converged to a local minimum of the
    function fmin or if Broyden's method can make no further progress. In this case try restarting
    from a different initial guess.
    const Int MAXITS=200;
    const Doub EPS=numeric_limits<Doub>::epsilon();
    const Doub TOLF=1.0e-8, TOLX=EPS, STPMX=100.0, TOLMIN=1.0e-12;
    Here MAXITS is the maximum number of iterations; EPS is the machine precision; TOLF
    is the convergence criterion on function values; TOLX is the convergence criterion on  $\delta\mathbf{x}$ ;
    STPMX is the scaled maximum step length allowed in line searches; and TOLMIN is used to
    decide whether spurious convergence to a minimum of fmin has occurred.
    Bool restrt, skip;
    Int i, its, j, n=x.size();
    Doub den, f, fold, stpmax, sum, temp, test;
    VecDoub fvcold(n), g(n), p(n), s(n), t(n), w(n), xold(n);
    QRdcmp *qr;
    NRfmin<T> fmin(vecfunc);           Set up NRfmin object.
    NRfdjac<T> fdjac(vecfunc);         Set up NRfdjac object.
    VecDoub &fvec=fmin.fvec;           Make an alias to simplify coding.
    f=fmin(x);                         The vector fvec is also computed by this
    test=0.0;                          call.
    for (i=0; i<n; i++)                Test for initial guess being a root. Use more
        if (abs(fvec[i]) > test) test=abs(fvec[i]);    stringent test than simply
    if (test < 0.01*TOLF) {              TOLF.
        check=false;
        return;
    }
    for (sum=0.0, i=0; i<n; i++) sum += SQR(x[i]);    Calculate stpmax for line searches.
    stpmax=STPMX*MAX(sqrt(sum), Doub(n));
    restrt=true;
    for (its=1; its<=MAXITS; its++) {    Ensure initial Jacobian gets computed.
        Start of iteration loop.
        if (restrt) {                    Initialize or reinitialize Jacobian and QR de-
            qr=new QRdcmp(fdjac(x, fvec));    compose it.
            if (qr->sing) throw("singular Jacobian in broydn");
        } else {                        Carry out Broyden update.
            for (i=0; i<n; i++) s[i]=x[i]-xold[i];     $\mathbf{s} = \delta\mathbf{x}$ .
            for (i=0; i<n; i++) {                 $\mathbf{t} = \mathbf{R} \cdot \mathbf{s}$ .
                for (sum=0.0, j=i; j<n; j++) sum += qr->r[i][j]*s[j];
                t[i]=sum;
            }
            skip=true;
        }
    }
}
```